

并发与同步

在 2000 年之前，CPU 的发展路线主要是提高 CPU 的运行频率。开发者无须做太多适配，就能使程序获得可观的性能提升。然而，通过提升频率以提升 CPU 性能的设计思路撞上了“能耗墙”，进一步提升频率会大大增加 CPU 的能量消耗，造成 CPU 发热严重，从而无法稳定使用。因此，单核 CPU 的性能提升很难再跟上摩尔定律的步伐。为了进一步提升 CPU 的性能，厂商开始尝试在一个 CPU 中添加多个物理核。2001 年 IBM 在 POWER 4 中加入了多核支持^[1]，标志着商用 CPU 正式进入多核时代。Intel 与 AMD^[2-3] 也在 2005 年加入了多核阵营。如今，市面上已经很难再找到单核的桌面级 CPU 了。除了桌面级 CPU，ARM 也在 2009 年发布了首个针对移动端的 ARM 架构多核 CPU Cortex-A5^[4]。图 9.1 展示了商用 CPU 的核心频率（Frequency）、核心数目（Number of Logical Cores）与单线程性能（Single-Thread Performance）逐年变化的趋势[⊖]。

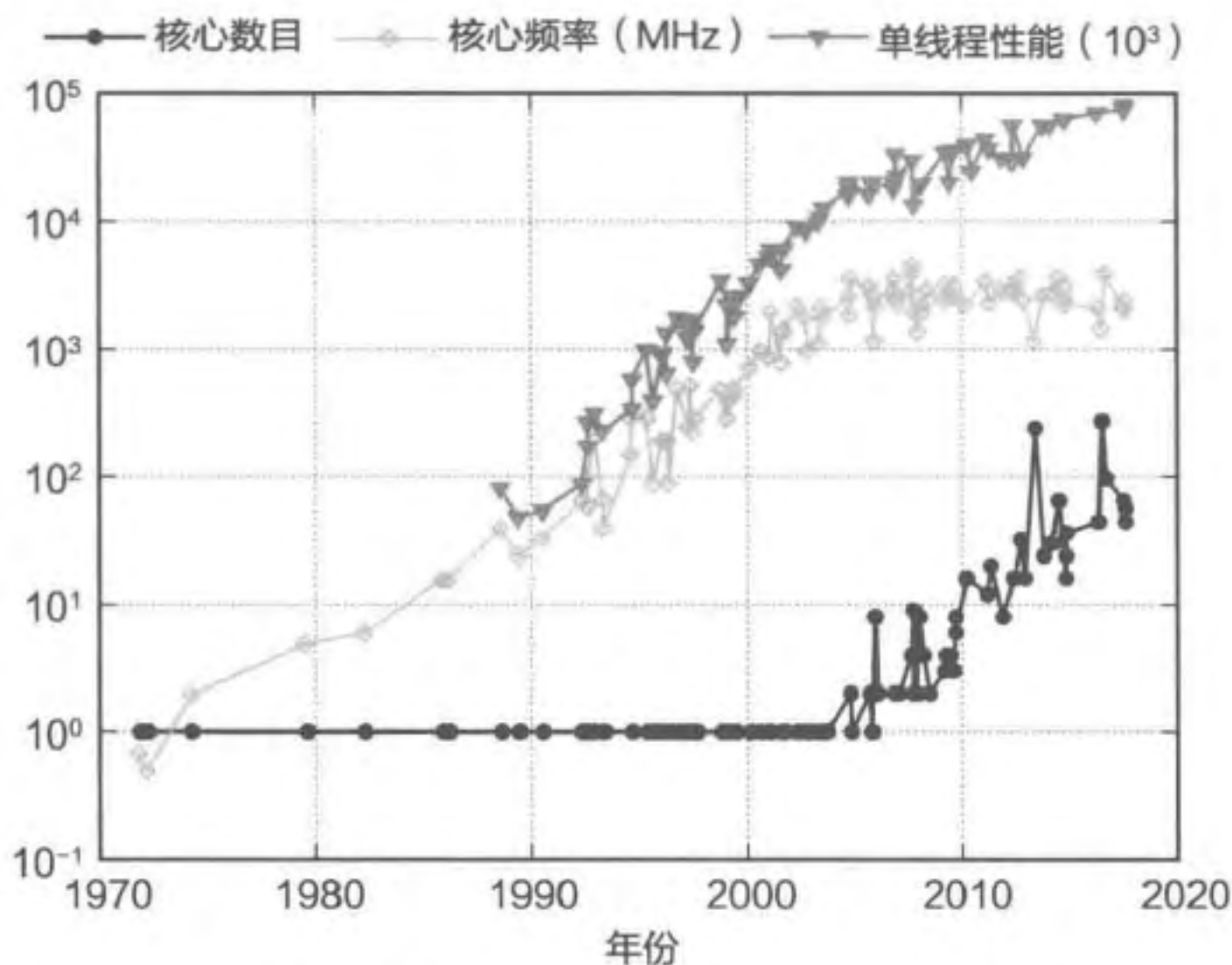


图 9.1 CPU 核心频率、核心数目与单线程性能的变化

⊖ 2010 年前的原始数据由 M. Horowitz、F. Labonte、O. Shacham、K. Olukotun、L. Hammond 和 C. Batten 收集，2010 年到 2017 年的数据由 K. Rupp 收集^[5]。其中单线程性能为使用 SPECint^[6] 测试集的跑分结果。

理想情况下，增加 CPU 核心数量后，应用程序应当获得呈比例的性能提升，然而现实却往往不尽如人意。为了充分利用多核处理器的能力，应用程序需要将待处理的任务进行划分，并将划分后的任务分配到不同线程中处理，从而在更短的时间内完成任务。比如，在科学计算中将矩阵依照行与列进行划分，在大数据处理中将海量数据依据核心数量进行划分，从而在多核上并行处理。但是，并行处理同一任务意味着对共享资源的并发访问，同一时刻可能存在多个线程对同一共享资源的访问与修改。为了保证共享资源状态的正确性，需要正确地在这些线程之间同步。

小知识：同步是多核带来的新问题吗？

不是。单核 CPU 也存在多个线程之间同步的需求。在处理器调度器的控制下，不同的线程交错执行，当多个线程访问共享资源时，也会出现冲突与正确性问题。在多核 CPU 中，任务可以被划分到运行在不同 CPU 核心上的线程同时处理。这使得线程之间的同步更加频繁，也为同步原语的实现带来了新的挑战。

9.1 同步场景

本节主要知识点

- 线程之间为何需要同步？
- 有哪些经典的同步场景？
- 这些同步场景之间的区别是什么？

本节将先以多线程计数器为例，为读者展现同步的重要性并引入一些基础概念。然后将列举从现实问题中抽象出来的与同步相关的四个典型场景，梳理不同场景下对于同步的需求。

9.1.1 一个例子：多线程计数器

代码片段 9.1 展示了一个简单的多线程计数器程序。在这段代码中，我们创建了两个线程，并使这两个线程频繁对同一个共享变量 a 进行 10^9 次自增操作（第 7 ~ 8 行）。在理想情况下，最终打印出来的变量 a 的值应当刚好等于 2×10^9 。然而，无论是在多核机器还是在单核机器上，这段代码打印出的最终结果都有很大概率远小于该值。读者可以将这段代码编译后在不同机器上运行来验证。如果在多核机器上测试，线程大概率会被调度到不同的核心执行。如果希望两个线程都在同一个核心上执行，可以通过在命令前使用 `taskset -c 0` 要求该程序只能在 0 号处理器上执行。

代码片段 9.1 多线程计数测试

```

1 #include <pthread.h>
2 #include <stdio.h>
3
4 unsigned long a = 0;
5 void *routine(void *arg)
6 {
7     for (int i = 0; i < 1000000000; i++)
8         a++;
9     return NULL;
10 }
11
12 int main(void)
13 {
14     pthread_t threads[2];
15     for (int i = 0; i < 2; i++)
16         pthread_create(&threads[i], NULL, routine, 0);
17     for (int i = 0; i < 2; i++)
18         pthread_join(threads[i], NULL);
19     printf("%lu\n", a);
20     return 0;
21 }

```

下面我们详细分析为何会出现这种违反常理的执行结果。自增操作其实对应了三条硬件汇编指令：首先读取自增对象的旧值至某一寄存器（ $\text{Reg} = a$ ），然后将该寄存器值的基础上执行自增操作（ $\text{Reg}++$ ），最后将寄存器中的自增结果存入共享的变量中（ $a = \text{Reg}$ ）。如果线程并行（即同时）执行在不同的处理器核心上，那么这三条硬件指令可能会交错执行。如图 9.2 所示，线程 1 与线程 2 同时执行在不同的处理器核心上。在时刻 T_0 ，线程 1 开始执行三条硬件指令中的第一条，将共享变量 a 的值读到处理器的某一寄存器 Reg 中。然后，在 T_1 时刻，线程 1 对该寄存器进行了自增操作。在同一时刻，线程 2 也开始执行这三条硬件指令。由于线程 1 对 a 的自增操作还未写回共享的变量，此时还存储在寄存器 Reg 中，因此线程 2 读取到的 a 的值还是旧值（还未自增）。最终导致在 T_2 与 T_3 时刻，两个线程向共享变量中存入的值相同，而线程 1 对于共享变量 a 的更新丢失（被线程 2 的更新覆盖）。最终线程 2 执行结束后， a 中的值只被增加了 1，而非开发者所期望的 2，发生了更新丢失。因此，在程序结束时，变量 a 的值远远小于 2×10^9 。发生更新丢失的本质原因是此时程序产生了数据竞争（Data Race）。数据竞争是指一个进程中多个线程同时访问同一个地址，且其中至少有一个是写操作。如果数据竞争发生，线程访存操作之间的顺序将是不确定的，导致程序的执行结果将不可预测，甚至出现错误。



图 9.2 计数问题：线程并行运行于不同的处理器核心上

在以上例子中，出现更新丢失时，两个线程同时运行在不同的处理器核心中。那么如果两个线程都运行在同一个处理器核心中，是否就不会出现更新丢失？很可惜，答案是否定的。由于抢占式调度可能会在任意时刻打断线程执行，因此这三条硬件指令还是会出现交错执行的情况。如图 9.3 所示，如果在线程 1 将更新的结果写回共享变量 a 之前，调度器打断了线程 1 的执行，并切换到线程 2 执行。此时线程 2 将读到共享变量 a 的旧值，并在此基础上执行自增操作。当调度器重新调度到线程 1 执行时，线程 1 将覆盖线程 2 的结果，导致更新丢失。可以从这个例子看出，在多线程程序中，如果多线程之间不加以协同，多个线程对共享数据的访问与修改很容易出现问题，导致与程序员预期的执行结果不符的情况发生。因此，线程之间需要使用合适的同步手段，保证多线程程序能够正确执行。为了正确、高效地解决这些同步问题，研究者抽象出一系列同步原语（Synchronization Primitive）供开发者使用。

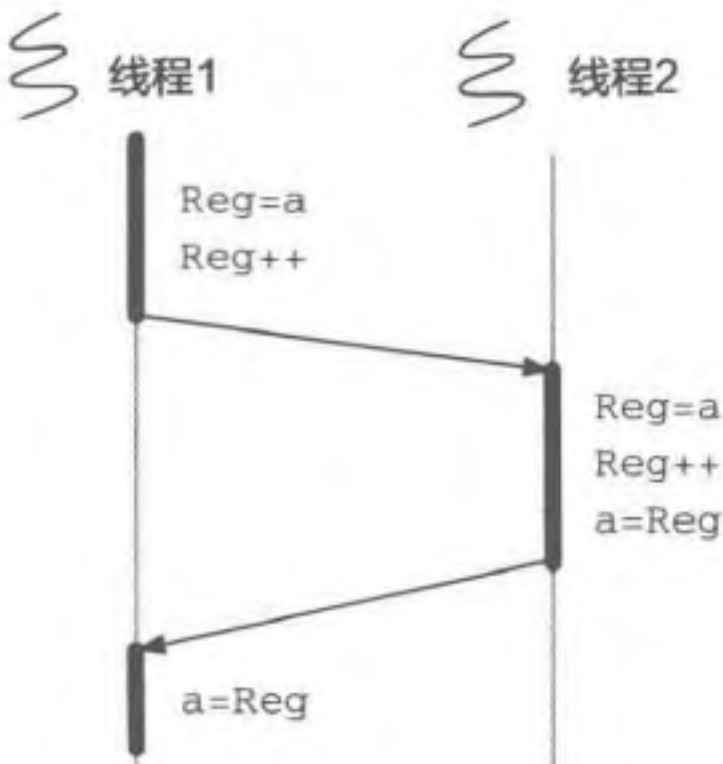


图 9.3 计数问题：线程运行于同一处理器核心上

9.1.2 同步的典型场景

多线程计数器仅仅是多线程应用面临的同步问题中一个简单的例子，在现实场景中多线程应用程序还会遇到各式各样的同步问题。究其本质，多线程应用需要正确协同多个不同的线程执行。本小节将介绍其中四个典型场景，这些场景涵盖了当前大部分多线程应用的同步需求。后续章节将以具体的场景为例，介绍如何分析具体问题中存在的同步需求，并使用合适的同步原语解决这些问题。

场景一：共享资源互斥访问

当多个线程需要同时访问同一共享数据时，应用程序需要保证互斥性才能避免数据竞争，从而确保程序的正确性。

如代码片段 9.2 所示，若线程 1（thread_1）与线程 2（thread_2）需要同时修改共享的 shared_var，则应用程序需要使用同步原语来避免同时修改 shared_var 造成数据竞争。前一节介绍的多线程计数就属于这类场景。

代码片段 9.2 场景一：共享资源互斥访问

```
1 int shared_var;
2 void thread_1(void)
3 {
4     shared_var = shared_var + 1;
5 }
6
7 void thread_2(void)
8 {
```

```
9  shared_var = shared_var - 1;
10 }
```

衍生场景一：读写场景并行读取

该场景为场景一的衍生场景。当多个线程同样需要访问同一共享数据，但多个线程只会读取共享数据时，允许这些线程并行（即同时）执行不会造成数据竞争，且能有效提升系统性能。

如代码片段 9.3 所示，若多个线程执行 reader 函数，其只读取共享的 shared_var 到局部变量 local_var。我们应该允许这些线程同时执行从而提升系统效率。而对于会修改数据的线程（执行 writer 函数），则需要保证没有其他线程在读写数据才能修改数据。

代码片段 9.3 衍生场景一：读写场景并行读取

```
1 int shared_var;
2 void reader(void)
3 {
4     local_var = shared_var;    // 只读取共享数据
5 }
6
7 void writer(void)
8 {
9     shared_var = shared_var + 1; // 修改共享数据
10 }
```

场景二：条件等待与唤醒

场景二与之前的场景不同，其不再关注如何协调线程对于共享资源的互斥访问，而是线程的执行顺序与条件（又称同步）。当线程需要等待某条件达成才能继续执行时，其应一直被阻塞直到条件达成。系统应提供对应的机制，在线程阻塞时调度到其他有计算需求的线程（即挂起该线程），避免浪费处理器以提升整体系统的效率。而在线程等待的条件达成后，系统应唤醒该阻塞等待（即挂起）的线程，让其能够继续执行。

如代码片段 9.4 所示，线程 2（执行 thread_2）现在需要等待线程 1（执行 thread_1）完成工作后再继续执行。操作系统应能够提供机制，在 thread_2 等待时调度到其他线程执行，最大限度地利用处理器的计算资源，并在线程 1 执行完毕之后（即条件达成）立刻唤醒线程 2 执行。

代码片段 9.4 场景二：条件等待与唤醒

```
1 int thread_1_state = UNFINISHED;
2 void thread_1(void)
3 {
4     doing_something();    // 完成当前线程的工作
5     thread_1_state = FINISH;
6     notify_thread_2();    // 通知线程 2 完成
```

```

7 }
8
9 void thread_2(void)
10 {
11     if (thread_1_state != FINISH)
12         wait();                // 等待线程 1 完成工作
13     doing_something();          // 完成线程 2 的工作
14 }

```

场景三：多资源协调管理

场景三是系统软件中十分常见的一种模式，其既包含对共享资源的管理，也包含协调多个不同线程的等待与执行。当多线程应用中存在多个资源可以被多个线程消耗或释放（生产）时，就需要协调这些线程有序获取资源或者等待。这里等待的条件就是有可用的资源，而唤醒的时机则是有线程释放（产生）了资源。

如代码片段 9.5 所示，当资源池 `shared_resources` 存在资源时，消费资源线程（执行 `consumer_thread` 函数）能成功获取资源并处理，否则将等待。生产资源线程（执行 `producer_thread` 函数）能向资源池生产（释放）新的资源，并且如果有线程在等待资源，可以通知这些线程获取资源并执行。

代码片段 9.5 场景三：多资源协调管理

```

1 struct resources *shared_resources;
2 void producer_thread(void)
3 {
4     produce(shared_resources); // 生产共享资源
5     notify_waiters();          // 通知等待的消费线程
6 }
7
8 void consumer_thread(void)
9 {
10     if (!has_resources(shared_resources))
11         wait();                // 等待资源
12     consume(shared_resources); // 消耗共享资源
13 }

```

9.2 同步原语

本节主要知识点

- ❑ 如何正确使用互斥锁、读写锁、条件变量与信号量？
- ❑ 针对不同的同步场景，如何选择合适的同步原语以满足同步需求？
- ❑ 不同同步原语有哪些不同及相似之处？
- ❑ 如何将多线程应用中的同步需求对应到同步场景，并使用合适的同步原语满足同步需求？